1.1 Architektura Transformer: Czym są tokeny, okno kontekstowe i mechanizm Attention?

Moduł 1: Fundamenty i Mechanizmy LLM

Krzysztof Rytlewski · 2026

Narodziny architektury Transformer

Przed 2017 rokiem dominujące metody przetwarzania języka naturalnego — RNN i LSTM — analizowały tekst sekwencyjnie, słowo po słowie. Miało to dwie istotne wady: wolne trenowanie na dużych zbiorach danych oraz trudność w utrzymaniu zależności między odległymi fragmentami tekstu.

Przełom nastąpił w 2017 roku, gdy badacze Google opublikowali pracę *„Attention Is All You Need"*. Zaproponowana architektura **Transformer** zastąpiła sekwencyjność przetwarzaniem **równoległym** — model analizuje wszystkie tokeny jednocześnie i ocenia relacje między nimi.

Transformer stał się fundamentem GPT-4, Claude, Gemini i Llama — zrozumienie tej architektury to zrozumienie silnika całej rewolucji AI.

Kluczowe składniki architektury Transformer

- **Embedding** — zamienia tokeny na wektory liczbowe; każdy token ma numeryczną reprezentację w przestrzeni wielowymiarowej
- **Positional Encoding** — dodaje informację o kolejności tokenów, bo równoległe przetwarzanie samo w sobie jej nie zachowuje
- **Multi-Head Attention** — kluczowy mechanizm decydujący, które tokeny są ze sobą powiązane i w jakim stopniu
- **Feed-Forward Layers** — gęste warstwy przetwarzające reprezentację każdego tokenu niezależnie od pozostałych
- **Layer Normalization i Residual Connections** — stabilizują trening głębokich sieci i zapobiegają znikaniu gradientów

Tokeny — jak model „widzi" tekst

Tokenizacja to proces zamiany tekstu na **tokeny** — fragmenty odpowiadające podjednostkom słów, całym słowom lub znakom interpunkcyjnym. Algorytm **BPE (Byte Pair Encoding)** analizuje częstotliwość par znaków w korpusie treningowym i tworzy słownik podjednostek (*subwords*).

Dla modelu tekst to ciąg **liczb całkowitych** (ID tokenów). Słowo `„programowanie"` może być jednym tokenem albo dwoma: `„program"` + `„owanie"` — zależy to od konkretnego modelu i jego słownika tokenizatora.

Tokenizacja odbywa się przed przekazaniem tekstu do modelu. Żaden model językowy nie operuje bezpośrednio na literach czy słowach.

Obserwacje praktyczne — tokeny i koszty

- Angielski tekst generuje średnio 1 token na ~4 znaki; języki słowiańskie, w tym **polski**, tokenizują się mniej efektywnie — więcej tokenów na słowo
- **Koszty API** naliczane są w tokenach, nie w słowach — sprawna tokenizacja promptów bezpośrednio wpływa na koszty przy dużej skali zapytań
- Tokeny mają **indeksy numeryczne (ID)** — dla modelu tekst jest ciągiem liczb, nie liter; sam tekst nie jest nigdy przesyłany do warstw Transformera
- Narzędzia do analizy: platform.openai.com/tokenizer lub pakiet `js-tiktoken` w Node.js — warto je znać przy optymalizacji kosztów

Okno kontekstowe — definicja i rozmiary

Okno kontekstowe to maksymalna liczba tokenów, jaką model może jednocześnie przetwarzać. Obejmuje zarówno wejście (prompt, historia rozmowy, dokumenty), jak i generowaną odpowiedź.

Analogia: robocza pamięć krótkotrwała człowieka — gdy tekst jest za długi, wcześniejsze fragmenty „wypadają" z pola widzenia.

Model	Okno kontekstowe
GPT-3.5 Turbo	16 384 tokenów (~12 tys. słów)
GPT-4o	128 000 tokenów (~96 tys. słów)
Claude 3.5 Sonnet	200 000 tokenów (~150 tys. słów)
Gemini 1.5 Pro	1 000 000 tokenów
Llama 3 (8B)	8 192 tokenów

Okno kontekstowe — praktyczne znaczenie

- **Długie dokumenty** (kod projektu, transkrypcje, raporty PDF) muszą zmieścić się w oknie lub wymagają strategii podziału — tzw. *chunking* stosowany w systemach RAG
- **Historia rozmowy** zajmuje miejsce w oknie; przy długich sesjach wczesne wiadomości mogą zostać „zapomniane" przez model — znikają z aktywnego kontekstu
- **Koszty** rosną proporcjonalnie do rozmiaru kontekstu — każdy token wejściowy jest płatny i wpływa na czas przetwarzania
- **Zjawisko „Lost in the Middle"** — modele gorzej przywoływują informacje umieszczone w środku długiego kontekstu niż na jego początku lub końcu

Mechanizm Attention — intuicja i Q, K, V

Mechanizm Attention odpowiada na pytanie: *„Na które tokeny powinienem zwrócić uwagę, analizując dany token?"*

Intuicja: W zdaniu *„Bank odmówił pożyczki, bo zbankrutował"* — słowo *zbankrutował* dotyczy banku, nie pożyczki. Attention pozwala modelowi dynamicznie powiązać każdy token z odpowiednimi tokenami w kontekście, niezależnie od odległości między nimi.

Każdy token transformowany jest w trzy wektory:

- **Query (Q)** — „Czego szukam?"
- **Key (K)** — „Co oferuję?"
- **Value (V)** — „Co przekazuję, gdy zostanę wybrany?"

Wzór Scaled Dot-Product Attention

Wynik uwagi obliczany jest jako ważona suma wektorów **V**, gdzie wagi wyznacza iloczyn skalarny **Q** z **K**, podzielony przez $\sqrt{d_k}$ i znormalizowany przez softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Dzielenie przez $\sqrt{d_k}$ stabilizuje gradienty przy dużych wymiarach wektorów. Nie trzeba znać wzoru na pamięć — ważne jest rozumienie **konceptu**: model uczy się dynamicznie decydować, które tokeny są najważniejsze dla danego tokenu w danym kontekście.

Multi-Head Attention — wiele perspektyw jednocześnie

Multi-Head Attention (MHA) uruchamia wiele równoległych mechanizmów uwagi, z których każdy specjalizuje się w innym typie relacji między tokenami. Wyniki wszystkich głowic są łączone i transformowane.

Analogia — panel ekspertów czytających ten sam tekst:

- **Głowica 1** — relacje gramatyczne: podmiot–orzeczenie
- **Głowica 2** — koreferencje zaimków
- **Głowica 3** — relacje semantyczne: synonimy, antonimy
- **Głowica 4** — relacje czasowe: kolejność zdarzeń

Duże modele mają dziesiątki głowic w każdej z wielu warstw, co tworzy bogatą, wielowymiarową reprezentację znaczenia każdego tokenu. GPT-4 wykorzystuje kilkadziesiąt głowic w każdej ze swoich warstw.

Encoder, Decoder i architektury hybrydowe

Architektura	Przykłady	Zastosowanie
Encoder-only	BERT, RoBERTa	Klasyfikacja tekstu, ekstrakcja informacji
Decoder-only	GPT-4, Claude, Llama	Generowanie tekstu, chatboty, kodowanie
Encoder-Decoder	T5, BART	Tłumaczenie, podsumowania

Wszystkie modele używane przez AI developera (GPT-4o, Claude, Llama 3) to architektura **decoder-only**. Generują tekst **autoregresywnie** — token po tokenie, każdorazowo na podstawie całego dotychczasowego kontekstu. Trening odbywa się równolegle, ale samo generowanie jest sekwencyjne od lewej do prawej.

Kluczowe wnioski

- LLM to statystyczne maszyny przewidujące następny token na podstawie poprzednich — nie „rozumieją" tekstu w ludzkim sensie, lecz modelują rozkłady probabilistyczne
- „Myślenie" modelu to wielowarstwowe przekształcenie reprezentacji wektorowych przez mechanizm uwagi
- Zrozumienie tokenów i okna kontekstowego pozwala świadomie projektować prompty i zarządzać kosztami wywołań API
- Mechanizm Attention działa wyłącznie w granicach **aktywnego okna kontekstowego** — informacje spoza niego dla modelu nie istnieją podczas generowania odpowiedzi